

VOGUE: VirtIO-GPU on Unikraft for Graphics Virtualization

Submission: #001

Abstract

Unikernels promise tiny images, millisecond boots, and a small attack surface, but every one of those wins comes from leaving things out: no shell, no user/kernel separation, no multi-process runtime, and — until now — no GPU. Meanwhile the workloads that most want a GPU, from graphics to LLM inference engines such as `llama.cpp`, are exactly the ones a unikernel cannot run today. The standard way to give a guest GPU access, VirtIO-GPU together with Mesa’s Venus driver, drags the entire Linux graphics stack into the guest: the kernel DRM/KMS subsystem, `libdrm`, and Mesa. That is precisely the bulk a unikernel exists to avoid.

This paper asks whether a *minimal* unikernel guest can reach a real GPU without importing Linux’s graphics stack. We present VOGUE, a VirtIO-GPU stack for the Unikraft unikernel. Our central observation is that a unikernel has exactly one address space and one purpose, so the GPU has exactly one client; the DRM multiplexing machinery that dominates the Linux path can therefore be removed entirely and replaced by a minimal exclusive-client driver. VOGUE is organized as four thin guest libraries that mirror, layer for layer, the much larger Linux tower, collapsing a roughly three-million-line graphics stack to about five thousand lines while speaking the identical VirtIO-GPU and Venus protocol that the host already understands.

We build three execution paths on this substrate — software-rendered 2D display, a minimal virgl 3D path, and Vulkan compute — and port `kmscube`, `glmark2`, `vkmark`, and upstream `llama.cpp` without modifying their sources. Following a strict evidence-gated methodology that never lets a passing check at one abstraction level stand in for a claim at another, we show that the guest image is 292 KB and boots in 10–11 ms, that graphics workloads render through our path, and that upstream `llama.cpp` runs GPU-accelerated inference over Venus on a Tesla V100, reaching 2045.8 tokens/s prefill and 139.3 tokens/s decode and serving real HTTP requests over lwIP. A performance gap to a Linux guest on the same Venus bridge remains, and we locate it precisely in our own Vulkan and driver code rather than in any missing GPU bridge. VOGUE thus shows that reaching the GPU need not mean “become Linux again.”

1 Introduction

A unikernel fuses a single application with just-enough operating system into one bootable image: one address space, no processes, booted straight by the hypervisor [6]. The payoff is concrete. Images are measured in hundreds of kilobytes rather than hundreds of megabytes; the system is ready in milliseconds rather than seconds; and the attack surface is only the code that was actually linked in. Every

one of these wins, however, comes from leaving things out. There is no shell, no user/kernel separation, and no multi-process runtime, and the ecosystem is young enough that many drivers and features have simply not been ported yet.

GPU support is one of those missing features, and the gap matters. Plenty of in-demand workloads need a GPU: graphics and rendering for desktop, GUI, and 3D visualization; LLM inference with engines such as `llama.cpp`; and a long tail of ML training, video transcoding, and scientific compute. On a unikernel today these workloads either cannot run at all or are forced onto the CPU. The natural response is to add GPU support — but the obvious way of doing so reintroduces exactly what the unikernel was built to avoid.

1.1 The dilemma

The standard recipe for giving a guest GPU access is VirtIO-GPU paired with Mesa’s Venus driver [9, 11]. On Linux that recipe requires the entire guest-side graphics stack: the kernel DRM/KMS subsystem, the GEM memory manager, `libdrm`, the Vulkan loader, and Mesa itself. A single Vulkan call falls through this tall tower before it ever leaves the guest. Importing that tower into a unikernel would mean re-importing the bulk a unikernel exists to shed — in effect, “becoming Linux again” just to reach the device.

Yet there is a seam in this picture. What actually crosses from guest to host is not the tower; it is a stream of `VIRTIO_GPU_CMD_*` commands and Venus-encoded Vulkan calls. The host — QEMU plus virglrenderer plus a real GPU driver — does not care *who* emits that stream. The same bytes work regardless of which guest produced them. This makes the guests interchangeable, and it raises the question this paper sets out to answer:

Can a minimal unikernel guest reach a GPU without importing Linux’s graphics stack?

Concretely: can we run GPU-accelerated graphics and LLM inference inside a unikernel without giving up its tiny image and instant boot?

1.2 VOGUE

We answer in the affirmative with VOGUE, a VirtIO-GPU stack built on the Unikraft library OS. The design turns a property of the unikernel into a simplification of the driver. Because a unikernel is one process with one purpose, the GPU device has exactly one client, so the DRM multiplexing, context arbitration, and resource locking that dominate the Linux path are unnecessary. We discard them and keep a minimal exclusive-client driver.

The result is a graphics substrate of roughly five thousand lines — against a Linux stack on the order of three million — arranged as four thin guest libraries that map directly onto the Linux layers they replace, while emitting the same

VirtIO-GPU and Venus protocol the host already speaks. On top of this substrate we build three execution paths (2D software display, a minimal virgl 3D path, and Vulkan compute) and port four upstream applications unmodified.

This paper makes the following contributions:

- **A dependency-collapse design.** We show that single-client ownership lets a unikernel replace the Linux DRM/Mesa tower, layer by layer, with a small chain of Unikraft libraries that reach the identical protocol seam (§3).
- **Three execution paths and unmodified ports.** We implement software 2D display, a deliberately minimal virgl 3D path, and a Vulkan compute path, and run `kmscube`, `glmark2`, `vkmark`, and upstream `llama.cpp` over them without source changes (§4).
- **An evidence-gated evaluation.** Following a methodology that refuses to let a pass at one layer imply success at another, we report a 292 KB image, 10–11 ms boot, working graphics paths, and `llama.cpp` GPU inference over Venus on a Tesla V100, while keeping the remaining performance gap explicit and located in our own code (§5).

2 Background

2.1 Unikraft: the library OS we build on

VOGUE is built on Unikraft, a framework that constructs a unikernel out of modular libraries [6]. The developer picks exactly what the application needs — a libc such as `musl`, a TCP/IP stack such as `lwIP`, a scheduler, the device drivers in use — and only those libraries are linked, with the single application, into one image that QEMU/KVM boots directly.

What you get is one address space and one purpose per image, with nothing linked in that was not asked for. What is *not* there is just as important: there is no Linux kernel, hence no `/dev/dri` and no kernel modules, and there are no processes, no `fork/exec`, and no dynamic loader. The consequence drives the rest of this paper: anything Linux hands an application for free — device nodes, mapping GPU memory, loading a driver at run time — VOGUE must either build itself or do without.

2.2 Why VirtIO-GPU

There are two roads to virtualizing a GPU: passing a physical device through to one guest (GPU passthrough), or exposing a paravirtual device that the host backs with its own driver (VirtIO-GPU) [11, 15]. We take VirtIO-GPU, for two reasons that matter specifically to a tiny unikernel.

First, it is a natural fit for tiny VMs. A single physical GPU can be *shared* across many guests — important for the lightweight GUI- and service-style workloads unikernels target — and the guest needs only a thin frontend, so the image stays unikernel-small and migratable. Second, it is cheap and thin to implement: VirtIO-GPU is just a standard virtio device, and Unikraft already has the virtio and PCI transport, which we reuse unchanged.

2.3 VirtIO-GPU and Venus

VirtIO-GPU is the paravirtual GPU device proper: it provides virtio queues, GPU resources, and fences for synchronization. On its own it carries 2D display commands and a 3D command channel. GPU *acceleration* rides on top of it through Venus [5, 9].

Venus, defined by Mesa and virglrenderer, takes a different approach from translating graphics APIs: rather than translate, it *serializes the Vulkan API itself* into a command stream. The guest encodes each Vulkan call’s arguments into a packed binary stream, ships that stream over VirtIO-GPU’s 3D submit channel, and the host replays it on its real Vulkan driver:

guest app (Vulkan) → Venus command stream → QEMU + virglrenderer → host Vulkan driver → GPU

The encoding is deliberately compact. Venus uses a PACKED layout with no inter-field alignment padding; pointer presence is signalled by a 64-bit word and array presence by a 32-bit count. The guest must serialize every field by hand and at exactly the right width, because the host decoder will silently misparse a stream that disagrees, without reporting an error. This is the modern path for GPU virtualization and the one VOGUE targets; crucially, the guest only *emits commands* — the real GPU work happens host-side.

2.4 What this costs a Linux guest

On Linux, that single Vulkan call falls through a tall stack before it leaves the guest at all. Table 1 lists the guest-side layers.

The key observation is that what crosses to the host is *not* this tower. It is only the stream of `VIRTIO_GPU_CMD_*` and Venus commands the bottom of the tower emits. The host does not care who produced them: the same stream is accepted from any guest, which makes guests interchangeable at this seam. A unikernel can therefore speak this protocol with only a fraction of the tower above it — and that fraction is exactly what VOGUE builds.

3 Design

3.1 Key principle: a single-client GPU driver

The design rests on one observation. A unikernel is one process with one purpose and one address space, so the GPU device has exactly *one* client. On Linux the GPU is a shared resource that several processes may use at once, which is why the DRM (Direct Rendering Manager) subsystem exists: it arbitrates multiplexed access, switching contexts, locking resources, and managing permissions. None of that is needed when there is only one client.

VOGUE therefore discards DRM multiplexing wholesale and replaces it with a minimal exclusive-client driver. Table 2 quantifies what this buys.

Two consequences follow immediately. With no DRM multiplexing logic to carry, the core driver is about 1,650 lines and stays auditable. With no user/kernel boundary,

Table 1: On Linux, four guest-side layers — none of which a unikernel wants — sit between the application and the paravirtual device.

Layer (top → device)	Owned by
<i>application</i>	the app
<i>Vulkan loader</i>	Linux graphics stack
<i>Mesa Venus ICD (userspace driver)</i>	Linux graphics stack
<i>libdrm</i>	Linux graphics stack
<i>kernel DRM virtgpu driver (+ DRM/KMS/GEM)</i>	Linux graphics stack
<i>virtio-gpu device</i>	the hypervisor

Table 2: Single-client ownership collapses the graphics stack by roughly 600× and shortens the command path to a direct virtqueue write.

Dimension	Linux virtio-gpu	VOGUE
<i>Graphics stack</i>	~3M LoC	~5K LoC
<i>Address space</i>	Kernel + user	Single
<i>GPU clients</i>	Multiple (needs arbiter)	Exclusive single client
<i>Command path</i>	app → ioctl → kernel → virtio	app → direct virtio ring

Table 3: The four guest layers of VOGUE.

Layer	Components
<i>L1 Application</i>	<code>kmscube / glmark2 / llama.cpp</code> (<i>unmodified</i>)
<i>L2 Compat</i>	<code>libukegl + libukswrender</code> (2D display) · <code>libukggml_vk</code> (Vulkan dispatch)
<i>L3 Venus</i>	<code>libukvenus</code> (Vulkan → Venus binary)
<i>L4 VirtIO</i>	<code>libukvirtio_gpu + libukdma</code> (→ Unikraft virtqueue → QEMU)

submitting a command is a direct function call onto the virtqueue rather than an `ioctl` that crosses into the kernel, removing a context switch from the hot path.

3.2 System architecture

VOGUE is organized into four layers, summarized in Table 3 and sketched against the host in Figure 1. Each layer has a precise role.

L1 — Application. Upstream application source, not a line modified, compiled directly into the unikernel image. Depending on the workload it uses EGL/GLES2 (2D display), Vulkan (compute), or the virgl encoder API (3D) of the layer below.

L2 — Compat. An API-compatibility layer that presents the surface an application expects on Linux and routes those calls into VOGUE. `libukegl` provides an EGL/GLES2/GBM/DRM stub whose `eglSwapBuffers` drives the software renderer and then issues 2D VirtIO commands; `libukswrender` is a 264-line CPU rasterizer that fills geometry into a DMA buffer; and `libukggml_vk` is a static Vulkan dispatch table that replaces the Linux Vulkan loader’s run-time `dlopen` of an ICD, letting `llama.cpp` route straight to L3. The components

differ entirely by path, and the 3D virgl path does not use this layer at all.

L3 — Venus. The Vulkan-command serialization layer. `libukvenus` turns Vulkan API calls into a Venus PACKED binary stream and is logically equivalent to Mesa’s Venus guest driver, without the rest of Mesa. It exists only on the Vulkan compute path; the 2D and virgl paths skip it.

L4 — VirtIO. The guest-side implementation of the VirtIO-GPU protocol and the single point where all three paths converge. `libukvirtio_gpu` packs each path’s commands into VirtIO-GPU wire format — 2D commands (`RESOURCE_CREATE_2D`, `TRANSFER_TO_HOST_2D`, `SET_SCANOUT`, `RESOURCE_FLUSH`), 3D/Venus commands (`CTX_CREATE`, `SUBMIT_3D`, `RESOURCE_CREATE_BLOB`, `RESOURCE_MAP_BLOB`), and the minimal virgl encoding in `virgl_encoder.c` — and `libukdma` provides the DMA-capable backing memory. The virtqueue itself (descriptor rings, doorbell, PCI BAR mapping) is internal transport pulled in through Unikraft’s virtio bus, not a separate layer.

On the host, the guest’s VirtIO-GPU commands pass through QEMU’s `virtio-gpu-gl-pci` device to `virglrenderer`,

Table 4: $\equiv$ same $\rightarrow$ replaced with a leaner VOGUE equivalent. The host bridge is shared, which is why VOGUE drops into existing hypervisors.

Layer	Linux VM		VOGUE Unikraft
L1 App	app (kmscube / llama.cpp)	$\equiv$	app (unmodified upstream)
L2 Compat	libEGL + libgbm + libdrm + Vulkan loader	$\rightarrow$	libukegl · libukswrender · libukggml_vk
L3 Venus	Mesa Venus driver	$\rightarrow$	libukvenus (same encoding, no Mesa)
L4 VirtIO	virtio-gpu + kernel virtio/PCI (/dev/dri ioctl)	$\rightarrow$	libukvirtio_gpu (direct virtqueue, no ioctl)
Host	QEMU virtio-gpu $\rightarrow$ virglrenderer $\rightarrow$ host GPU driver	shared	QEMU virtio-gpu $\rightarrow$ virglrenderer $\rightarrow$ host GPU driver

which calls the host’s real Vulkan or OpenGL driver to do the actual GPU work. Because that host bridge is left

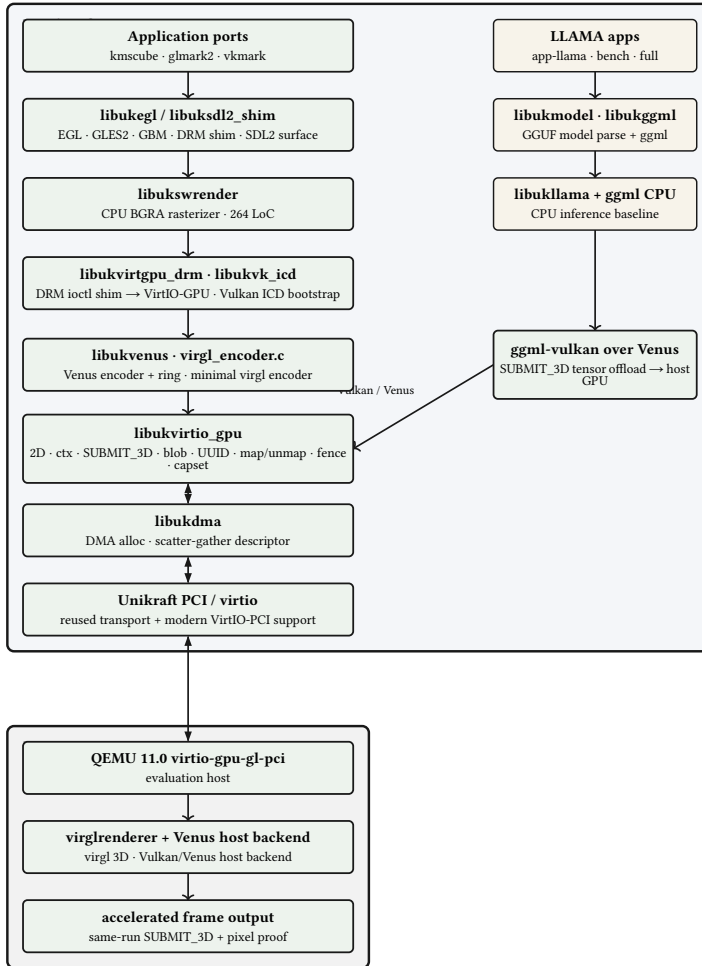


Figure 1: The VOGUE guest software stack. The left column is the 2D display path (CPU software render through libukswrender) together with the Vulkan/Venus acceleration substrate; the right column is the llama.cpp compute side-chain that reaches real GPU inference on the host. Both paths converge on libukvirtio_gpu and the Unikraft PCI/virtio transport, then cross to the shared host bridge (QEMU virtio-gpu-gl-pci $\rightarrow$ virglrenderer $\rightarrow$ host GPU driver).

untouched, VOGUE is compatible with existing hypervisors out of the box. The thinness shows up in the artifact: the guest image is **292 KB** (a Linux kernel is 9.2 MB — 31× larger) and boot takes **10–11 ms** (Linux plus initramfs takes 857–861 ms — 80× slower), because there is no kernel module system, no DRM subsystem, no init process, and no dynamic loading to wait on.

3.3 VOGUE versus Linux, layer by layer

Every VOGUE layer has a Linux counterpart; some are logically equivalent ($\equiv$) and some are replaced by a leaner equivalent ($\rightarrow$), as shown in Table 4. At L1 the application is unchanged. At L2, Linux ships real libEGL, libgbm, and libdrm plus a Vulkan loader that dlopens an ICD at run time; VOGUE substitutes stubs (libukegl, libukswrender) and a static dispatch table (libukggml_vk), eliminating dynamic loading. At L3, Mesa’s Venus driver serializes Vulkan into Venus binary, and libukvenus does the same encoding without depending on the rest of Mesa. At L4, Linux enters the kernel through a /dev/dri ioctl before reaching the virtqueue, whereas libukvirtio_gpu drives the Unikraft virtqueue directly, with no ioctl path. The host side — QEMU, virglrenderer, and the host GPU driver — is shared and unchanged.

3.4 Three execution paths

The four layers compose differently for the three workloads VOGUE supports; Table 5 places each path against the layers. The differences are the point: the paths reach the GPU (or not) by different routes, and L2 and L3 are absent on some of them by design.

The **2D display** path never touches the GPU: libukswrender draws pixels into memory on the CPU and libukvirtio_gpu pushes them to QEMU with 2D display commands. The **Vulkan compute** path is the full stack: libukggml_vk (L2) catches llama.cpp’s Vulkan calls, libukvenus (L3) serializes them, and SUBMIT_3D carries the result to virglrenderer, which runs the matrix work on the host GPU and writes results back — with no image ever displayed.

The **3D virgl** path is the interesting one: it jumps from L1 directly to L4, skipping both L2 and L3, and it does so for two independent reasons. It skips L2 because a compatibility shim needs a public API boundary to stand on. Vulkan has one (vkCreateDevice, vkQueueSubmit), so

Table 5: Three execution paths across the four layers.

	2D Display	3D Rendering	Vulkan Compute
<i>L1 App</i>	kmscube SW · glmark2	kmscube virgl	llama.cpp
<i>L2 Compat</i>	libukswrender (CPU)	— (direct encoder)	libukggml_vk (Vk dispatch)
<i>L3 Venus</i>	—	—	libukvenus (→ Venus)
<i>L4 VirtIO</i>	TRANSFER/SET_SCANOUT/FLUSH	SUBMIT_3D (virgl)	SUBMIT_3D (Venus)
<i>GPU</i>	none (CPU only)	host GPU → buffer	host GPU → buffer (no display)

libukggml_vk can intercept there; but the Gallium command stream is a Mesa-*internal* concept, not a public API. The virgl application produces Gallium binary itself, calling `uk_virgl_encode_create_surface()` and friends from the L4 encoder header directly, so there is no seam at which a compat layer could insert itself. It skips L3 because virgl speaks the Gallium protocol, not Venus, so libukvenus is simply inapplicable; and because VOGUE’s virgl encoder is deliberately tiny — three commands (create-surface, set-framebuffer, clear), about eighty lines, enough to clear the screen to a color. A complete virgl encoder would have to cover the entire OpenGL state machine (Mesa’s runs to thousands of lines), which would violate the small-substrate goal below. At this size the encoder belongs inside L4, and the corresponding evidence goals are transport-level proofs, not a claim of full OpenGL acceleration. A full virgl serializer at L3 is left to future work.

3.5 Design goals

The design follows five goals, the last of which is as much a methodological commitment as a technical one.

- **DG1 — Standards-based.** Follow the OASIS VirtIO-GPU specification so the guest is compatible with QEMU, crosvm, and cloud-hypervisor; invent no private protocol.
- **DG2 — Small trusted substrate.** Keep the guest graphics stack around 5,000 lines so the trusted computing base is fully auditable, which matters for security-sensitive unikernel deployments.
- **DG3 — Application source compatibility.** Run kmscube, llama.cpp, and the rest from *unmodified* upstream source, which forces the compat shims to faithfully emulate the EGL, GLES2, GBM, and DRM surfaces these apps expect.
- **DG4 — Progressive validation.** Validate native tests (no QEMU) → software render → QEMU transport → accelerated render can be localized to a specific layer.
- **DG5 — Evidence-gated claims.** Strictly distinguish claims at different abstraction levels. A `gfx.kmscube.sw` pass (software rendering) does *not* imply virgl or GPU acceleration; a `proto.real-driver` pass (static ABI check) does *not* imply that QEMU executes the stream; a host Linux baseline is *not* evidence of Unikraft runtime performance. This discipline is what keeps the evaluation honest, and we carry it through §5.

4 Implementation

The implementation follows the narrative arc of the design: we built the core driver, validated the simplest path first, then enabled the GPU, and along the way confronted how exacting the protocol specifications are. This section covers the driver stack (§4.1) and the application ports (§4.2).

4.1 The driver stack

4.1.1 VirtIO-GPU: the core driver

libukvirtio_gpu implements the full VirtIO-GPU protocol in about 1,650 lines of core driver logic, owning the device exclusively. The simplest thing it does is the 2D display pipeline that brings up kmscube and glmark2, and it does so without touching the GPU at all: the CPU draws pixels, the driver pushes them to QEMU, binds the resource to the display, and waits.

Setup happens once. `RESOURCE_CREATE_2D` asks QEMU to allocate a 2D resource of a given resolution and pixel format, and `RESOURCE_ATTACH_BACKING` hands QEMU the physical page list of the guest’s DMA buffer; from then on the guest and host share that memory. Each frame then repeats five steps, shown in Figure 2: the CPU rasterizer fills the DMA buffer, `TRANSFER_TO_HOST_2D` tells QEMU to read guest memory into the host resource, `SET_SCANOUT` binds the resource to a virtual display, `RESOURCE_FLUSH` asks QEMU to present it, and a fence wait blocks until QEMU has finished the frame so the CPU does not overwrite a buffer QEMU is still reading.

Fences are how the guest learns a command finished: it submits a fence command carrying a unique ID, QEMU completes commands in submission order and writes the fence ID back to a guest-readable location, and the guest

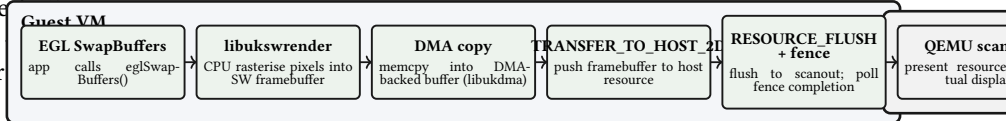


Figure 2: The per-frame 2D display pipeline. libukswrender rasterises the frame into a DMA-backed buffer on the CPU; `TRANSFER_TO_HOST_2D` pushes it into the host resource, `SET_SCANOUT` binds the resource to the virtual display, and `RESOURCE_FLUSH` (carrying a completion fence) presents it. The guest waits on that one fence before starting the next frame, so it never overwrites a buffer QEMU is still reading. No GPU is involved on this path.

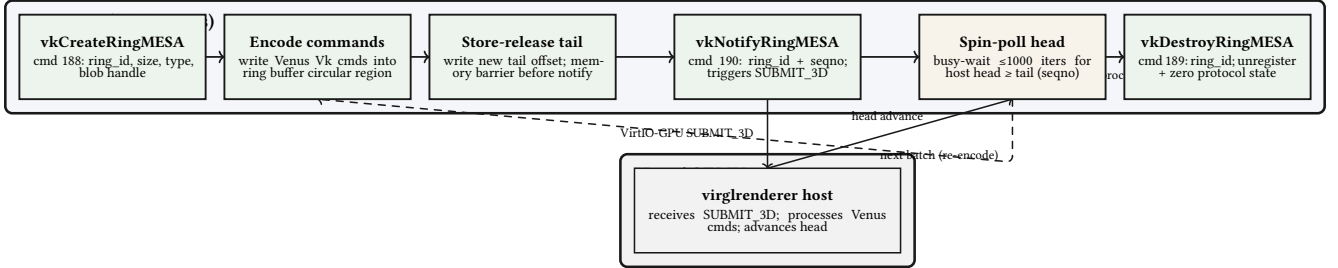
Table 6: The Venus ring-buffer protocol. **Top**: the ring header lives in a HOST3D_GUEST blob in host-visible memory; cache-line-separated slots hold head, tail, and status so guest and host can synchronise without false sharing, zero-copy. **Bottom**: the command sequence. The guest registers the ring with `vkCreateRingMESA` (cmd 188), encodes Venus commands into the circular data region, publishes a new tail with a store-release, and calls `vkNotifyRingMESA` (cmd 190) to trigger one `SUBMIT_3D`; the host advances head as it consumes commands while the guest spin-polls, and `vkDestroyRingMESA` (cmd 189) tears the ring down.

Batching many commands per notification amortises the `SUBMIT_3D` cost.

Ring buffer header layout (HOST3D_GUEST blob, host-visible memory)

head (offset 0)	tail (offset 64)	status (offset 128)	extra (offset 192)
host writes head; guest reads to detect consumed commands	guest writes tail to advance ring; host reads to fetch commands	ring state (ready / busy / error); polled by guest spin-wait	reserved / future protocol extension field

Mesa ring-buffer protocol command sequence



spin-polls until it sees that ID. This in-order guarantee is precisely what enables an optimization. The VirtIO-GPU specification mandates in-order processing, so:

Naive: TRANSFER → fence₁ → wait → FLUSH → fence₂ → wait
Optimized: TRANSFER → FLUSH → fence → wait

Because TRANSFER is submitted before FLUSH, waiting on the FLUSH fence already implies the TRANSFER completed; the transfer needs no fence of its own, saving one round-trip wait per frame. With this path, `kmscube` reaches about 484 fps in software and `glmark2`'s scene-clear run works.

4.1.2 Venus: GPU acceleration

Venus is the GPU-acceleration extension of VirtIO-GPU. `libukvenus` serializes each Vulkan call into a PACKED binary stream and ships it through `SUBMIT_3D` to the host's `virglrenderer`, which decodes and executes it on the real GPU. Two mechanisms work together to make this efficient, illustrated in Table 6.

Shared memory (L4). A blob resource gives the guest a pointer into host-visible memory. `RESOURCE_CREATE_BLOB` allocates a host-visible region of type `HOST3D_GUEST`, `RESOURCE_MAP_BLOB` maps it into the guest address space through the PCI SHM BAR, and `RESOURCE_UNMAP_BLOB` releases it. Where an ordinary transfer copies (guest writes → DMA copy → host reads), a blob lets the guest write where the host reads directly — zero copy. This shared region is the backing store for the Venus ring buffer.

Ring buffer (L3). The ring is built on the blob resource with a cache-line-separated layout: head at offset 0 (advanced by the host), tail at offset 64 (advanced by the guest), status at offset 128, and the circular, power-of-two data region at offset 192. Separating the control fields by 64 bytes — one CPU cache line — keeps the guest's tail

updates and the host's head updates from causing false sharing. The guest registers the ring with `vkCreateRingMESA` (command 188), batches many serialized Vulkan commands into the data region, publishes a new tail with a store-release so the command data is visible before the tail, and calls `vkNotifyRingMESA` (command 190) to trigger a single `SUBMIT_3D`; it then polls head (up to 1,000 iterations) until the host catches up. `vkDestroyRingMESA` (command 189) tears the ring down. Each `SUBMIT_3D` is a full virtqueue operation — write descriptors, ring the doorbell, reap the response — so amortizing many commands into one notification is what makes the path practical.

The Venus PACKED encoding is unforgiving, and getting it exactly right was a substantial part of the work. Fields carry no inter-field alignment padding, a 64-bit field is not 8-byte aligned in the stream, pointer presence is a 64-bit word, and array presence is a 32-bit count — a distinction that, when confused, produces a stream the host silently mis-parses. We validated the entire protocol against a software-only fake backend that mimics `virglrenderer`'s responses, so encoding correctness is tested independently of QEMU integration. The end-to-end results — `virgl` pixel-correct frames, and `llama.cpp` GPU inference at roughly 160 tok/s decode in the same-run appliance — were then obtained on real QEMU with a real host GPU, keeping the two kinds of claim separate.

4.2 Applications: small ports, real workloads

VOGUE provides the missing system pieces; the application code stays upstream. Table 7 lists the four ports and what each proves. The line counts are VOGUE-owned glue — ABI shims and bridge code — not the vendored application body.

Table 7: Application ports. LOC is VOGUE-owned glue, not the application body.

App	Role	VOGUE glue	What it proves
kmscube	minimal graphics demo	593 LOC / 3 ABI shims	unchanged app draws inside our Unikraft system
glmark2	OpenGL app bring-up	155 LOC / 3 ABI shims	a larger GL app can start and run on our path
vkmark	Vulkan app bring-up	62 LOC / 2 Vulkan shims	a Vulkan app can start and load its scenes
llama.cpp	real compute workload	upstream bridge + dispatch	LLM inference runs inside Unikraft

Table 8: Libraries on the Vulkan compute path. libukvenus is mostly *generated* from the upstream Venus protocol and Vulkan API metadata; the hand-written part is the Unikraft-side ring and glue code.

Library	Role	Source LOC
libukggml_vk	static ggml-vulkan dispatch	2,146
libukvenus	(mostly generated) Venus encoder/protocol + ring glue	96,690
libukvirtgpu_drm	Linux virtgpu DRM ioctl shim	638
libukvirtio_gpu	VirtIO-GPU frontend + virgl/controlq	2,246
libukvk_icd	Vulkan ICD bootstrap shim	235

Table 9: Evaluation host. The host bridge is identical to Linux’s; only the guest is replaced.

Layer	Setup
VM monitor	QEMU 11.0.1
GPU device	virtio-gpu-gl-pci, blob=true, venus=true
Host bridge	Venus-enabled virglrenderer
Host GPU	Tesla V100-SXM2-16GB
Guest	Unikraft appliance, one image per purpose
Rule	same-run artifact or no claim

4.2.1 Why compute came before graphics

Although graphics is the more intuitive GPU workload, compute was the practical first target to reach the *real* GPU, because the Vulkan compute pipeline is a much shorter path than the graphics pipeline. Compute has one main create structure (`VkComputePipelineCreateInfo`), one shader stage, a simple data-in / data-out flow, and no screen output. Graphics requires `VkGraphicsPipelineCreateInfo` plus ten or more fixed-function state blocks, at least vertex and fragment stages that must all match, and management of render targets and presentation. Compute fits a one-purpose appliance; graphics expects a window and display stack. After proving the graphics basics on the 2D and virgl paths, compute was the faster route to a real GPU workload.

4.2.2 The compute path reaches the real GPU

For `llama.cpp` the full chain is: upstream `llama.cpp` bench/server $\rightarrow$ `ggml-vulkan` $\rightarrow$ `libukggml_vk` static dispatch $\rightarrow$ `libukvenus` encoder and Venus protocol $\rightarrow$ `libukvirtgpu_drm` + `libukvirtio_gpu` $\rightarrow$ QEMU `virtio-gpu-gl` + `virglrenderer` $\rightarrow$

host Vulkan driver on a Tesla V100. Table 8 lists the libraries this path adds and their source sizes.

The size of `libukvenus` is almost entirely generated code: the encoder and protocol tables are produced from the upstream Venus protocol and Vulkan API metadata, and only the Unikraft-side ring and glue code is hand-written. This is how a single guest library can cover the breadth of the Vulkan API surface while remaining within the spirit of the small-substrate goal — the trusted, hand-audited surface is small even though the generated surface is large.

5 Evaluation

Our evaluation follows the evidence-gated discipline of DG5: we report what a same-run artifact actually shows, and no more. The guiding rule is “same-run artifact or no claim.” We first describe the host, then summarize the current stage of each path, then give graphics and `llama.cpp` numbers, and finally the server.

Table 10: Current stage by path.

What we checked	Goal	Status today
<i>Graphics basics</i>	graphics apps can open buffers and draw	virgl sends 3 real 3D submits in our small test
<i>Vulkan basics</i>	create devices, talk to Venus, start a real app	DRM shim, ICD setup, Venus command path, and vkmark scene loading all run
llama.cpp <i>GPU</i>	upstream compute uses the real GPU through Venus	ggml-vulkan inference runs inside Unikraft on the Tesla V100
llama.cpp <i>HTTP</i>	GPU workload is usable as a service	/health endpoint served over lwIP

Table 11: Graphics results. Software rendering, app bring-up, and benchmark claims are reported separately.

Workload	Current number / result	What we can say
glmark2.sw	142.82 fps in the 120-frame scene-clear run	subset benchmark, not full suite
kmscube <i>virgl</i>	3 virgl-submitted frames, 3 SUBMIT_3D commands	one small clear-frame proof
vkmark	10 startup scenes load through our Vulkan app path	no Unikraft FPS yet

Table 12: llama.cpp throughput (tokens/s). pp512 = prompt ingestion, tg128 = decode.

Same model / GPU path	Prefill pp512	Generate tg128
<i>CPU Unikraft</i>	9.4	7.4
<i>VOGUE Unikraft + Venus</i>	2045.8	139.3
<i>Linux VM + Venus</i>	4948.17	323.64
<i>Bare-metal Vulkan</i>	5586.91	239.16

5.1 Evaluation host

We evaluate on the same host bridge a Linux guest would use, but with a much thinner Unikraft guest. Table 9 lists the setup.

5.2 Current stage

Table 10 states, for each path, the goal and what runs today. The phrasing is deliberately literal because the rows mean different things: some prove software rendering, some prove transport, and some prove real GPU compute.

5.3 Graphics results

Table 11 gives the graphics numbers we can stand behind. We keep software rendering, application bring-up, and full-benchmark claims separate: glmark2.sw is a subset benchmark, the kmscube virgl result is a single small clear-frame proof, and vkmark only demonstrates that the Vulkan app starts and loads scenes.

5.4 llama.cpp: compute works, a performance gap remains

The headline systems result is that upstream llama.cpp runs GPU-accelerated inference inside a unikernel over Venus. Table 12 compares the same model and GPU path across four environments, in tokens/s; pp512 is 512-token prompt ingestion (prefill) and tg128 is 128-token decode.

Three things stand out. GPU offload is a large jump over CPU-only Unikraft — prefill rises from 9.4 to 2045.8 tokens/

s. A Linux guest over the *same* Venus bridge is still faster, which tells us the gap is not in the host bridge. Because the host side is shared and the Linux guest beats us on it, the remaining difference must live in our own Vulkan library and driver code — and that is exactly where we direct future tuning.

5.5 The server works

llama.cpp also runs as an HTTP service inside the unikernel, over lwIP. Table 13 gives the current server facts. The endpoints are live and the service handles real requests; the numbers come from a small burst and are an honest “it works,” not a tuned throughput claim.

5.6 Where the next speedup should come from

The evaluation also tells us where to look next, and notably the answer is *our code*, not a missing GPU bridge. The Linux VM beats VOGUE on the same Venus bridge (323.64 vs 139.3 tg128 decode tokens/s), which points at time spent in our Vulkan call path; server throughput is low (25.53 decode tokens/s in an 8-request burst measured at concurrency 1), which calls for guest SMP and split server threads; model loading copies too much (Vulkan server load takes 5935.26 ms with use_mmap=false), which calls for an mmap-capable model path; and graphics still lacks broad scene FPS, since vkmark loads ten scenes but yields no Unikraft FPS yet. These are next-step performance and breadth targets, not bring-

Table 13: llama.cpp server over lwIP. The service works; tuning it under real load is future work.

Server fact	Current result
<i>HTTP liveness</i>	/health, /v1/models, /completion return 200
<i>Slots configured</i>	4
<i>Throughput run</i>	8 requests at concurrency 1
<i>Prompt throughput</i>	146.14 tokens/s during prompt ingestion
<i>Decode throughput</i>	25.53 tokens/s during token generation
<i>Time to first token</i>	4.98 s
<i>Request rate</i>	0.159 requests/s in this small burst

up blockers — the prototype reaches the GPU and runs real workloads; the work that remains is making it fast and broad.

6 Related Work

Unikernels and library OSes. VOGUE builds on the unikernel tradition. MirageOS argued that single-purpose cloud services can be built as type-safe library OSes with small, specialized images [8], and Unikraft generalized the idea with a micro-library architecture and Kconfig composition [6], which is the substrate VOGUE extends. Drawbridge explored the opposite trade-off, preserving a rich Windows personality inside a library OS [13]; VOGUE instead implements the *smallest* graphics personality its workloads need. The exokernel line exposed low-level resources to applications [4], and Arrakis and IX split control-plane policy from data-plane I/O so applications can bypass kernel mediation on the fast path [2, 12]. VOGUE applies the same fast-path principle narrowly to graphics: the VirtIO-GPU data path becomes a library-level call while the hypervisor and device protocol remain the protection boundary.

Lightweight virtualization. Firecracker shows that microVMs deliver fast startup and strong isolation for serverless workloads [1]. VOGUE is complementary — it concerns what runs *inside* a minimal VM when the appliance needs a GPU, asking whether the guest graphics stack can stay light enough to keep the unikernel’s advantages.

GPU virtualization. VirtIO-GPU is our device target, specified by OASIS [11] and implemented by QEMU [15]. virglrenderer translates guest Gallium command streams to host OpenGL [17], Venus carries Vulkan over VirtIO-GPU [5, 9], and rutabaga/gfxstream serves ChromeOS and Android graphics virtualization [3], with vhost-user-gpu externalizing the backend for isolation [14]. These define host- and device-side mechanisms; VOGUE’s contribution is the *guest* side — a small Unikraft library stack that speaks the same protocol and runs selected applications, including same-run Vulkan execution of upstream llama.cpp, without Linux DRM or Mesa.

Minimal graphics stacks. Linux DRM/KMS [7], Mesa [10], GBM, EGL, and GLES form the commodity direct-rendering path. VOGUE replaces this with a compatibility boundary rather than a complete reimplementaion; its CPU rasterizer is intentionally smaller and less general than software renderers such as llvmpipe or OSMesa. The trade-off is explicit: VOGUE gains auditability and a tiny footprint but defers shader and compiler coverage until a full virgl path exists. Prior Unikraft porting work has focused on network and storage services [16]; VOGUE extends the pattern to GPU workloads by factoring each port into reusable shims and accounting for every stub through its evidence rows.

7 Future Work

The bring-up phase is over; what remains is performance and breadth, and the evaluation already points at where to spend effort. Table 14 lists the priorities. The highest-priority items address that we measured the server only at concurrency 1: adding guest SMP and more vCPUs, and tuning the llama.cpp server’s threads and slots, move it from “it works” to “it handles real load.” Next, the gap to a Linux guest on the same Venus bridge directs us to cut time spent in our own Vulkan call path, and to load the model with less copying (the current path runs without mmap or huge pages). Finally, matched Linux graphics baselines and small scene-render tests would turn the graphics results into a fair comparison with real frame numbers.

Beyond these immediate items, the longer-term value is a reusable method rather than any single benchmark number: continue turning the VirtIO-GPU/Venus stack analysis into reusable guest libraries, keep the one-image-one-purpose ports reproducible, and extend from today’s demos and the llama.cpp server to more graphics and GPU workloads.

8 Conclusion

We set out to answer whether a minimal unikernel guest can reach a real GPU without importing Linux’s graphics stack, and VOGUE shows that it can. The key is that a unikernel has exactly one GPU client, which lets us discard DRM multiplexing and replace a roughly three-million-line Linux

Table 14: From a working prototype to performance: the next targets.

Priority	Next step	Why it matters
P0	Add guest SMP and more vCPUs	the server was only measured at concurrency 1
P0	Tune <code>llama.cpp</code> server threads and slots	move from “it works” to real load
P1	Reduce time in our Vulkan call path	Linux on the same Venus bridge is still faster
P1	Load the model with less copying	the model path still runs without <code>mmap</code> or huge pages
P1	Add matched Linux graphics baselines	make graphics results a fair comparison

tower with about five thousand lines of Unikraft libraries that speak the identical VirtIO-GPU and Venus protocol the host already understands.

In doing so we made three things concrete. We analyzed the GPU path, breaking VirtIO-GPU down into driver, library, graphics, and compute pieces and mapping each onto its Linux counterpart. We showed a thin Unikraft design in which small guest libraries reach display, Vulkan, and Venus without importing Linux, in a 292 KB image that boots in 10–11 ms. And we proved real workloads: we ported `kmscube`, `glmark2`, `vkmark`, and upstream `llama.cpp` unmodified, ran graphics demos through our path, and served `llama.cpp` GPU inference over the real GPU on a Tesla V100. A performance gap to a Linux guest on the same Venus bridge remains — but because the host bridge is shared, that gap lives in our own code, not in any missing piece of the GPU path. Reaching the GPU, it turns out, need not mean becoming Linux again.

References

- [1] Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. 2020. Firecracker: Lightweight Virtualization for Serverless Applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2020. 419–434.
- [2] Adam Belay, George Prekas, Ana Klimovic, Samuel Grossman, Christos Kozyrakis, and Edouard Bugnion. 2014. IX: A Protected Dataplane Operating System for High Throughput and Low Latency. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2014. 49–65.
- [3] ChromiumOS contributors. 2024. Rutabaga: gfxstream graphics abstraction layer. Retrieved from https://crosvm.dev/doc/rutabaga_gfx/index.html
- [4] Dawson R. Engler, M. Frans Kaashoek, and James O’Toole. 1995. Exokernel: An Operating System Architecture for Application-Level Resource Management. In *Proceedings of the Fifteenth ACM Symposium on Operating Systems Principles (SOSP)*, 1995. 251–266. <https://doi.org/10.1145/224056.224076>
- [5] KhronosGroup/Mesa contributors. 2024. venus-protocol: Mesa Venus Vulkan over VirtIO wire-format headers.
- [6] Simon Kuenzer, Vlad-Andrei Bădoiu, Hugo Lefeuvre, Sharan Santhanam, Alexander Jung, Gauthier Gain, Cyril Soldani, Costin Lupu, Ștefan Teodorescu, Costi Răducanu, Cristian Banu, Laurent Mathy, Răzvan Deaconescu, Costin Raiciu, and Felipe Huici. 2021. Unikraft: Fast, Specialized Unikernels the Easy Way. In *Proceedings of the 16th European Conference on Computer Systems (EuroSys)*, 2021. <https://doi.org/10.1145/3447786.3456248>
- [7] Linux kernel contributors. 2024. Direct Rendering Manager (DRM/KMS) subsystem. Retrieved from <https://www.kernel.org/doc/html/latest/gpu/>
- [8] Anil Madhavapeddy, Richard Mortier, Charalampos Rotsos, David Scott, Balraj Singh, Thomas Gazagnaire, Steven Smith, Steven Hand, and Jon Crowcroft. 2013. Unikernels: Library Operating Systems for the Cloud. In *Proceedings of the 18th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2013. 461–472. <https://doi.org/10.1145/2451116.2451167>
- [9] Mesa contributors. 2024. Venus: Vulkan over VirtIO. Retrieved from <https://docs.mesa3d.org/drivers/venus.html>
- [10] Mesa contributors. 2024. Mesa 3D graphics library. Retrieved from <https://mesa3d.org/>
- [11] OASIS Technical Committee. 2023. *Virtual I/O Device (VIRTIO) Version 1.3*. Retrieved from <https://docs.oasis-open.org/virtio/virtio/v1.3/virtio-v1.3.html>

- [12] Simon Peter, Jialin Li, Irene Zhang, Dan R. K. Ports, Doug Woos, Arvind Krishnamurthy, Thomas Anderson, and Timothy Roscoe. 2014. Arrakis: The Operating System is the Control Plane. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2014. 1–16.
- [13] Donald E. Porter, Silas Boyd-Wickizer, Jon Howell, Reuben Olinsky, and Galen C. Hunt. 2011. Rethinking the Library OS from the Top Down. In *Proceedings of the 16th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2011. 291–304. <https://doi.org/10.1145/1950365.1950399>
- [14] QEMU Project. 2024. vhost-user-gpu: External VirtIO-GPU backend. Retrieved from <https://www.qemu.org/docs/master/interop/vhost-user-gpu.html>
- [15] QEMU Project. 2026. virtio-gpu — QEMU Documentation. Retrieved from <https://www.qemu.org/docs/master/system/devices/virtio-gpu.html>
- [16] Unikraft community. 2024. Unikraft application catalog. Retrieved from <https://github.com/unikraft/catalog>
- [17] virglrenderer contributors. 2024. virglrenderer: A virtual 3D GPU library. Retrieved from <https://gitlab.freedesktop.org/virgl/virglrenderer>